



Proactive Model Adaptation Against Concept Drift for Online Time Series Forecasting

Lifan Zhao

Shanghai Jiao Tong University
Shanghai, China
mogician233@sjtu.edu.cn

Yanyan Shen

Shanghai Jiao Tong University
Shanghai, China
shenyy@sjtu.edu.cn

Abstract

Time series forecasting always faces the challenge of concept drift, where data distributions evolve over time, leading to a decline in forecast model performance. Existing solutions are based on online learning, which continually organize recent time series observations as new training samples and update model parameters according to the forecasting feedback on recent data. However, they overlook a critical issue: obtaining ground-truth future values of each sample should be delayed until after the forecast horizon. This delay creates a temporal gap between the training samples and the test sample. Our empirical analysis reveals that the gap can introduce concept drift, causing forecast models to adapt to outdated concepts. In this paper, we present PROCEED, a novel proactive model adaptation framework for online time series forecasting. PROCEED first estimates the concept drift between the recently used training samples and the current test sample. It then employs an adaptation generator to efficiently translate the estimated drift into parameter adjustments, proactively adapting the model to the test sample. To enhance the generalization capability of the framework, PROCEED is trained on synthetic diverse concept drifts. Extensive experiments on five real-world datasets across various forecast models demonstrate that PROCEED brings more performance improvements than the state-of-the-art online learning methods, significantly facilitating forecast models' resilience against concept drifts. Code is available at <https://github.com/SJTU-DMTai/OnlineTSF>.

CCS Concepts

• Information systems → Data stream mining; • Computing methodologies → Online learning settings.

Keywords

Time series forecasting; Online learning; Concept drift

ACM Reference Format:

Lifan Zhao and Yanyan Shen. 2025. Proactive Model Adaptation Against Concept Drift for Online Time Series Forecasting. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.1 (KDD '25)*, August 3–7, 2025, Toronto, ON, Canada. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3690624.3709210>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
KDD '25, Toronto, ON, Canada.

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-1245-6/25/08
<https://doi.org/10.1145/3690624.3709210>

KDD Availability Link:

The source code of this paper has been made publicly available at <https://doi.org/10.5281/zenodo.14554743>.

1 Introduction

Time series forecasting has been a prevalent task in numerous fields such as climate [4], energy [5, 26], retail [7], and finance [33]. Recent years have witnessed a surge of deep learning-based forecast models [11, 14, 19, 23, 28, 36] that take past time series observations to predict values in the next H steps, where H is called *forecast horizon*. Due to the dynamic nature of the environment, latent *concepts* that influence observation values (e.g., social interest [2], stock market sentiment [17]) often change over time. This ubiquitous phenomenon is known as *concept drift* [13]. In the presence of concept drift, future test data may not follow a similar data distribution as the historical training data, causing degradation in forecasting performance [30, 33].

Online learning is commonly used to mitigate the effects of concept drift. The key consideration is to continually transform the newly observed time series into a set of training samples and adjust model parameters by minimizing the forecast errors on the new training samples. In addition to the standard fine-tuning technique, recent works [24, 32] proposed more advanced model adaptation techniques, which focus on how to effectively adapt to recent data by utilizing *forecasting feedback* (e.g., errors or gradients) on the new training samples. Among them, FSNet [24] monitors the gradients in previous fine-tuning, transforms them into parameter adjustments, and tailors a new forecast model to the current training samples. OneNet [32] is an online ensembling network that generates ensemble weights to combine two forecast models and dynamically adjusts the ensemble weights and the models' parameter weights according to the forecast errors. However, existing online learning methods overlook the fact that the ground truth of each prediction is not available immediately but is delayed after the forecast horizon.

As illustrated in Fig. 1, an online learning task inherently has a H -step *feedback delay* in time series forecasting, resulting in a temporal gap (at least H steps) between available training samples and the test sample. Formally, at each online time t , the current horizon window Y_t , which has not been observed yet, has an overlap with $Y_{t-H+1}, \dots, Y_{t-1}$. These samples cannot provide complete forecasting feedback and supervision for the performant, prevailing time series forecasting models that predict H -step values directly [19, 23, 31]. Hence, the available training samples for online learning at time t is $\mathcal{D}_{t-} = \{(X_{t'}, Y_{t'}) \mid t' \leq t - H\}$, which has a temporal distance to the test sample (X_t, Y_t) . The temporal gap issue suggests that addressing concept drift by fitting the forecast

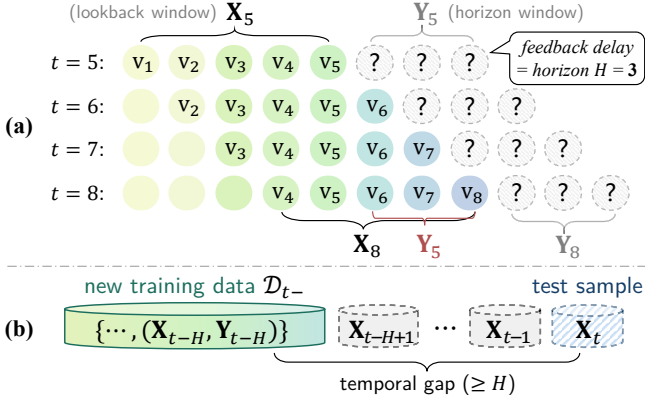


Figure 1: (a) Example of feedback delay when $H = 3$. At on-line time $t = 5$, we use observation values $X_5 = \{v_1, \dots, v_5\}$ to forecast future values $Y_5 = \{v_6, v_7, v_8\}$, while the ground truth is known until $t = 8$ and the feedback of forecasting performance arrives with 3-step delay. (b) A temporal gap always exists between new training samples D_{t-} and the test sample X_t , where concept drift may occur.

model to the recent training samples may be insufficient, since concept drift may also occur over the H -step temporal gap between the recent training samples and the current test sample.

To investigate the impact of this temporal gap on forecasting performance, we examine two online learning strategies using real-world time series datasets (detailed in Section 3). The first strategy fine-tunes model parameters using the latest available training sample at time t , which is (X_{t-H}, Y_{t-H}) . The second strategy [24, 32] omits the feedback delay and adapts the model using (X_{t-1}, Y_{t-1}) , which is infeasible in practice due to future information leakage. The empirical results show that the average forecast error of the first strategy is approximately double that of the second strategy, indicating that even the latest observed time series pattern differs from the test sample notably. Moreover, the performance gap between the two strategies becomes more significant with a longer horizon, suggesting that the concept drift becomes more pronounced over longer time intervals. These findings highlight the presence of a substantial concept drift between the practical training samples and the test sample, limiting the effectiveness of current online learning techniques that adapt model parameters to potentially outdated concepts and leave the concept drift unresolved.

In this paper, we aim to answer the question: how can we effectively adapt the forecast model to each test sample and boost online time series forecasting performance against ongoing concept drifts? A straightforward idea is to estimate a sketch of the latent concept of the test data and customize model parameters for the estimated concept. However, it is challenging to train a model adapter (e.g., a mapping function) that directly generates model parameters based on the estimated concept of the test sample. First, test data may bear a new concept that is out of the distribution of the historical time series. There is no experience in generating optimal parameters for such an out-of-distribution concept. Second, a parameterized model adapter that maps concepts to model parameters (or their updates) essentially lies in a parameter space of $\mathbb{R}^{m \times n}$, where m is

the concept embedding dimension and n is the number of model parameters. Since advanced time series forecasting models often involve a high quantity of parameters (e.g., 1 million [19, 23]), the model adapter is hard to optimize and easily suffers from overfitting due to the huge parameter size and limited test data.

To address these issues, we propose **PROCEED**, a **PROaCtive** **model** **ad**aptation framework that responds to the concept drift before forecasting the test sample. In scenarios where optimal model parameters change with time, we posit that such parameter changes are affected by the drift across a latent concept space, i.e., there may exist learnable relationships between the direction and degree of concept drift and those of parameter changes. In light of this, we propose to infer parameter changes based on concept drift, rather than directly generating a whole model. Specifically, PROCEED begins with a forecast model that has learned new training samples. Given a test sample, PROCEED exploits latent features that encapsulate the concept drift between the training samples and the test sample. In response to the undergoing concept drift, PROCEED generates parameter adjustments via bottleneck layers, which yield a compact set of adaptation coefficients that rescale all model parameters. In this way, our solution eschews direct mapping from concept drift to per-parameter adjustments, instead opting for a more nuanced and efficient strategy. We expect the adapted model with rescaled parameters to approach the optimum for the test sample. To enhance the generalization ability of PROCEED, we shuffle historical data to synthesize diverse concept drifts, on which we train PROCEED to learn the relationships between concept drifts and desirable parameter adjustments. When confronted with a re-occurring concept drift that has been learned among the synthetic concept drifts, PROCEED can tailor the model to the test sample.

The contributions of our work can be summarized as follows.

- We highlight that time series forecasting has an inherent feedback delay issue, and we provide an empirical analysis to demonstrate the presence of concept drift between the newly acquired training sample and the test sample. Empirically, such concept drift is more significant with a longer forecast horizon, while it remains unresolved in existing online learning methods.
- We propose **PROCEED**, a proactive model adaptation framework for online time series forecasting under concept drift. PROCEED first estimates the latent representation of concept drift and then customizes parameter adjustments that adapt the parameters to the concept of the test sample in advance of online prediction.
- Extensive experiments on five real-world time series datasets demonstrate that PROCEED remarkably reduces the average forecast error of different forecast models by a large margin of 21.9%. Moreover, PROCEED outperforms existing online model adaptation methods by an average of 10.9% and keeps high efficiency.

2 Definitions and Problem

Consider N distinct variates ($N \in \mathbb{N}^+$). Let $\mathbf{v}_t \in \mathbb{R}^N$ denote the N -dimensional observation values at time t . A time series is a sequence of observation vectors in time order, i.e., $\mathbf{v}_1, \mathbf{v}_2, \dots$.

DEFINITION 1 (TIME SERIES FORECASTING). Let L be the lookback window size and H be the horizon window size, where $L, H \in \mathbb{N}^+$. At time t , a forecast model $\mathcal{F}$ parameterized by θ takes the past

observations $\mathbf{X}_t = [\mathbf{v}_{t-L+1}, \dots, \mathbf{v}_t] \in \mathbb{R}^{N \times L}$ as input features to predict the future H -step values, denoted as $\mathbf{Y}_t = [\mathbf{v}_{t+1}, \dots, \mathbf{v}_{t+H}] \in \mathbb{R}^{N \times H}$. The training objective is to optimize the model parameters θ such that the loss function $\|\hat{\mathbf{Y}}_t - \mathbf{Y}_t\|_2^2$ is minimized, where $\hat{\mathbf{Y}}_t \in \mathbb{R}^{N \times H}$ denotes the predicted values in the horizon window.

Typically, in multi-step forecasting where $H > 1$, there are two primary strategies to generate the predictions $\hat{\mathbf{Y}}_t$ at each time t . The first strategy performs *iterative forecasting*. That is, the model forecasts one step ahead and uses its prediction to forecast the next step, repeating this process for the entire horizon. Despite its simplicity, this strategy suffers from significant error accumulation over long horizons [31]. The second strategy adopts *direct forecasting* where the model makes all the predictions $\hat{\mathbf{Y}}_t$ simultaneously. Note that the two strategies require different output modules or layers in the forecast model. Recent works [23, 31] have shown that direct forecasting tends to outperform the iterative method, particularly for longer forecast horizons. Hence, this paper adopts the direct forecasting strategy where a sample $\mathbf{X}_t$ is considered valid for training if all the H -step values in $\mathbf{Y}_t$ are known [9].

In online forecasting scenarios, time series data are observed sequentially. Due to the dynamic nature of real-world processes, underlying data distributions are subject to constant change. Consequently, a forecast model trained on historical data may encounter difficulties when confronted with new, evolving patterns. This issue is referred to as the *concept drift* challenge, which can substantially affect model performance over time [13]. To mitigate concept drifts, it is crucial to adapt the forecast model continuously to assimilate new concepts presented in the incoming time series. Formally, the online model adaptation problem is defined as follows.

DEFINITION 2 (ONLINE MODEL ADAPTATION). *At time t , online model adaptation activates a model adapter $\mathcal{A}$ that produces adapted model parameters $\hat{\theta}_t$ based on available observations $\{\mathbf{v}_1, \dots, \mathbf{v}_t\}$, where $\hat{\theta}_t$ is expected to be close to the optimal parameters θ_t^* . Then, we forecast $\mathbf{Y}_t$ by $\mathcal{F}(\mathbf{X}_t; \hat{\theta}_t)$.*

At a high level, existing model adaptation methods [9, 24, 32] select some observed data $\mathcal{D}_{t-} \subset \{(\mathbf{X}_{t'}, \mathbf{Y}_{t'}) \mid t \leq t - H\}$ as training samples and utilize the forecasting feedback (e.g., forecasting errors and gradients w.r.t. $\mathcal{D}_{t-}$) to update the model parameters. Their adapted parameters tend to align with the patterns or concepts present in $\mathcal{D}_{t-}$. However, it is crucial to recognize that the concepts present in $\mathcal{D}_{t-}$ may not necessarily reflect that of the test sample $(\mathbf{X}_t, \mathbf{Y}_t)$ due to the horizon time span H . To this end, the model optimized on $\mathcal{D}_{t-}$ might still be susceptible to concept drift, potentially resulting in suboptimal predictions at time t (see details in Section 3).

In what follows, we provide empirical analysis that illustrates the presence of concept drift between $\mathcal{D}_t$ and the test sample $(\mathbf{X}_t, \mathbf{Y}_t)$, revealing the limitations of existing model adaptation techniques.

3 Empirical Analysis and Motivation

In this section, we present an empirical study examining how concept drift between newly acquired training data and test data affects the effectiveness of existing online learning approaches for different time series forecasting models. This analysis highlights the need for developing proactive strategies to adapt forecast models.

3.1 Datasets

Following prior works [24, 32], we use five real-world time series datasets, including ETTh2, ETTm1, Weather, ECL, and Traffic. We provide their detailed descriptions in Appendix B.1 and statistical information in Table 7. We also adhere to the evaluation settings of FSNet [24] and OneNet [32], where each dataset is chronologically divided into training/validation/test sets by the ratio of 20:5:75.

3.2 Baseline Variants

We consider the following three time series forecasting models.

- **FSNet [24]:** FSNet is built upon a TCN [3] backbone and further develops an advanced updating structure that facilitates fast adaptation to concept drift.
- **OneNet [32]:** OneNet is an ensemble model that dynamically combines two forecasters, one focused on temporal dependence and one focused on channel dependence. We follow its official implementation where the two forecasters are built upon FSNet.
- **PatchTST [23]:** PatchTST is one of the state-of-the-art time series forecasters, which models temporal patterns by Transformer.

We pre-train the forecast models on the training set and then perform online learning across the validation set and test set. For each of the models, we compare two online learning techniques below.

- **Practical:** At each time t , before forecasting $\mathbf{Y}_t$, we perform predictions on $\mathbf{X}_{t-H}$ and use the Mean Square Error (MSE) between the ground truth $\mathbf{Y}_{t-H}$ and the prediction $\hat{\mathbf{Y}}_{t-H}$ to update the model by one-step gradient descent.
- **Optimal:** At each time t , before forecasting $\mathbf{Y}_t$, we assume the ground truth $\mathbf{Y}_{t-1}$ is available without any delay. We calculate the previous forecast loss on the last sample $\mathbf{X}_{t-1}$ (i.e., MSE between $\mathbf{Y}_{t-1}$ and $\hat{\mathbf{Y}}_{t-1}$) to update the model by one-step gradient descent.

It is important to notice that the *Optimal* strategy uses the most relevant and recent information for the prediction on the test sample $\mathbf{X}_t$. However, it is infeasible in practice as it requires knowledge of future data. The performance gap between this ideal strategy and the *Practical* strategy reveals the presence of concept drift that is not adequately addressed by existing online learning techniques. To maintain a fair comparison, both strategies fine-tune all the model parameters using one training sample each time.

3.3 Evaluation Metrics

Following previous works [24, 31, 35], we evaluate the forecasting performance by two commonly used metrics as defined below:

$$\text{Mean Square Error (MSE)} = \frac{1}{N|T_{\text{test}}|} \sum_{t \in T_{\text{test}}} \|\hat{\mathbf{Y}}_t - \mathbf{Y}_t\|_2^2,$$

$$\text{Mean Absolute Error (MAE)} = \frac{1}{N|T_{\text{test}}|} \sum_{t \in T_{\text{test}}} \|\hat{\mathbf{Y}}_t - \mathbf{Y}_t\|_1,$$

where T_{test} represents all time steps of the test set and $\hat{\mathbf{Y}}_t \in \mathbb{R}^{N \times H}$ denotes the predicted result at time t . A lower error indicates higher forecasting performance. Each experiment is repeated 3 times with different random seeds and we report the average results. Due to the page limit, we provide the MAE results in Appendix D.1.

Table 1: MSE results of two online learning strategies for time series forecasting. We report the MAE results in Appendix D.1.

Model	Dataset H	ETTh2			ETTm1			Weather			ECL			Traffic		
		24	48	96	24	48	96	24	48	96	24	48	96	24	48	96
FSNet	Optimal	0.687	0.846	1.087	0.584	0.724	0.706	0.786	1.039	1.107	5.668	5.811	5.979	0.362	0.464	0.501
	Practical	3.079	4.247	6.213	0.763	0.923	1.003	0.875	1.338	1.755	6.143	9.435	12.941	0.458	0.517	0.574
	Δ_{MSE}	348%	402%	472%	31%	27%	42%	11%	29%	59%	8%	62%	116%	26%	11%	15%
OneNet	Optimal	0.532	0.609	0.993	0.173	0.173	0.189	0.379	0.397	0.415	2.074	2.201	3.089	0.364	0.400	0.438
	Practical	2.965	4.892	8.257	1.335	1.547	1.040	0.861	1.182	1.484	9.757	10.847	15.932	0.462	0.544	0.591
	Δ_{MSE}	457%	703%	732%	672%	794%	452%	128%	198%	257%	370%	393%	416%	27%	36%	35%
PatchTST	Optimal	1.664	2.596	4.367	0.245	0.240	0.236	0.546	0.565	0.547	4.085	4.699	5.602	0.358	0.341	0.348
	Practice	2.092	3.434	5.770	0.455	0.598	0.674	0.735	0.979	1.261	4.143	4.762	5.791	0.376	0.380	0.398
	Δ_{MSE}	26%	32%	32%	86%	149%	185%	35%	73%	130%	1%	1%	3%	5%	11%	15%

[‡] We augment FSNet and OneNet with RevIN [16] for better performance. The results differ from those in the original paper. We explain the reasons in Appendix B.2.

Additionally, we quantify the performance gap between the *Practical* and the *Optimal* by the following equation:

$$\Delta_{\text{MSE}} = \frac{\text{MSE}_{\text{Practical}} - \text{MSE}_{\text{Optimal}}}{\text{MSE}_{\text{Optimal}}} \times 100\%.$$

3.4 Key Observations

Table 1 shows the MSE results of two strategies. We have the following major observations. **First**, the *Practical* strategy performs worse than *Optimal* by an average of 107% on different models. This significant difference suggests that the most recently observed pattern in $(\mathbf{X}_{t-H}, \mathbf{Y}_{t-H})$ substantially differs from $(\mathbf{X}_{t-1}, \mathbf{Y}_{t-1})$ and fails to reflect the concept of the test sample $(\mathbf{X}_t, \mathbf{Y}_t)$. In other words, the adapted forecast models are still vulnerable to the concept drift caused by the feedback delay issue. **Second**, we can observe the general tendency for the performance gap to become more significant as the forecast horizon increases. This could be attributed to the increasing temporal distance between $\mathbf{X}_{t-H}$ and $\mathbf{X}_t$. **Third**, OneNet demonstrates the best performance when utilizing the *Optimal* strategy. However, its effectiveness drastically diminishes in all the cases when employing the *Practical* strategy, even becoming the worst on the ETTm1 and Traffic datasets. The reason is that OneNet is an ensemble model with a large number of parameters, which increases the overfitting risk on the new training samples. **Fourth**, PatchTST, recognized as the leading time series forecasting model, reports much smaller Δ_{MSE} than the other models. When using the *Practical* strategy, PatchTST outperforms all the other models. However, it still falls short of the performance achieved by OneNet under the *Optimal* strategy. To sum up, there is still considerable room for performance improvement if we can address the possible concept drift between each test sample and the latest available training data during online learning.

4 The Proposed Framework: PROCEED

In this section, we first discuss our idea and provide an overview of the proposed PROCEED solution. We then elaborate on two major steps, namely concept drift estimation and proactive model adaptation. Finally, we describe the end-to-end training scheme that improves the model’s adaptation ability against concept drifts.

4.1 Solution Overview

The ultimate goal of PROCEED is to close the gap between the newly acquired training data and the test sample and boost performance against concept drift caused by the feedback delay issue. To achieve this, a simple solution is to extract the latent concepts from all historical samples and learn a mapping function between each concept and optimal model parameters *w.r.t.* the historical sample. As such, one can extract concept from each test sample and use the mapping function to directly generate possibly optimal parameters. However, the online phase may include new concepts that are out of the historical data distribution. The simple solution can fail in this case, since it has not learned the relationship between out-of-distribution (OOD) concepts and corresponding optimal parameters.

To address the problem, we propose to map *concept drifts* to *parameter shifts*. We assume that the direction and degree of the drift over the concept space can reflect a possible direction and magnitude of parameter shifts over the parameter space, informing how to make an adaptation. Specifically, given a model that fits new training samples, PROCEED exploits latent features from the training samples and the current test sample, estimates the undergoing concept drift, and accordingly predicts parameter shifts.

Following existing online model adaptation methods [24, 32], we use one training sample for model updating at each time t , *i.e.*, $\mathcal{D}_{t-} = (\mathbf{X}_{t-H}, \mathbf{Y}_{t-H})$. With the model updated on $\mathcal{D}_{t-}$, we estimate the concept drift between $\mathcal{D}_{t-}$ and $\mathbf{X}_t$, predict potential shifts in the parameter space, and accordingly make parameter adjustments. Formally, our PROCEED solution consists of four key steps at each time t as listed below.

- (1) **Online Fine-tuning.** Given a forecast model $\mathcal{F}$ parameterized by θ_{t-H-1} , we redo forecasting by $\hat{\mathbf{Y}}_{t-H} = \mathcal{F}(\mathbf{X}_{t-H}; \theta_{t-H-1})$. Next, we use the forecast error $\|\hat{\mathbf{Y}}_{t-H} - \mathbf{Y}_{t-H}\|_2^2$ to update θ_{t-H-1} into θ_{t-H} by gradient descent. The subscript of θ indicates that the parameters have fit $(\mathbf{X}_{t-H}, \mathbf{Y}_{t-H})$.
- (2) **Concept drift estimation.** Given $\mathcal{D}_{t-}$ and the test sample $\mathbf{X}_t$, we feed them into two concept encoders $\mathcal{E}$ and $\mathcal{E}'$ that extract concept representations denoted as $\mathbf{c}_{t-H} \in \mathbb{R}^{d_c}$ and $\mathbf{c}_t \in \mathbb{R}^{d_c}$, respectively. Then, we estimate the hidden state of the concept drift between $\mathbf{X}_{t-H}$ and $\mathbf{X}_t$ by $\delta_{t-H \rightarrow t}$, where $\delta_{t-H \rightarrow t} = \mathbf{c}_t - \mathbf{c}_{t-H}$.

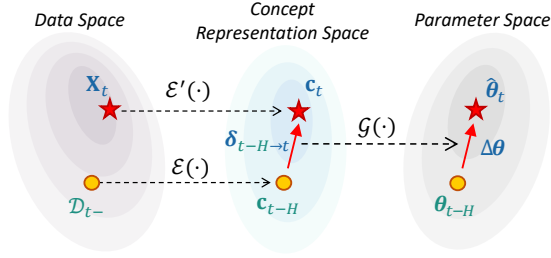


Figure 2: Illustration of the mapping from time series data change to parameter shift. Concept encoders $\mathcal{E}$ and $\mathcal{E}'$ encode time series into concept representations. Adaptation generator $\mathcal{G}$ decodes the conceptual difference into parameter shift.

- (3) **Proactive model adaptation.** Given the estimated concept drift $\delta_{t-H \rightarrow t}$ and the parameters θ_{t-H} , we employ an adaptation generator $\mathcal{G}$ to generate parameter shifts $\Delta\theta$, adjusting θ_{t-H} into θ_t as illustrated in Fig. 2.
- (4) **Online forecasting.** Finally, the adapted model yields predictions by $\hat{Y}_t = \mathcal{F}(X_t; \hat{\theta}_t)$. At the next time $t+1$, the parameters will be reset to θ_{t-H} .

The rationale of our solution is that we can synthesize diverse concept drifts based on historical data to train our model adapter. Fig. 3 shows an example of the historical training data including four samples with their concepts denoted by $c_1, \dots, c_4$ respectively. Our idea is to shuffle the order of samples and train our model adapter on the synthetic concept drifts between each pair of samples. Though the concepts of the future test samples (i.e., $c_5, \dots, c_7$) are out-of-distribution, the concept drift patterns $\delta_{4 \rightarrow 6}$ and $\delta_{5 \rightarrow 7}$ are similar to $\delta_{1 \rightarrow 4}$, $\delta_{2 \rightarrow 3}$ which have been learned in training data. Given such recurring concept drifts, we can generate appropriate parameter shifts by experience.

4.2 Concept Drift Estimation

In the literature, there are numerous concept drift detection methods [13] that estimate the degree of concept drift to decide when to adapt the model. The degree can be estimated by the changes in forecast error [8], distribution distance [10], prediction uncertainty [37], *etc.* Nevertheless, the degree as a scalar in $\mathbb{R}$ is not informative and cannot indicate how to adapt the model. Therefore, we propose to model a high-dimensional representation vector in $\mathbb{R}^{d_c}$ that characterizes both the degree and the direction of the concept drift, where d_c is the representation dimension.

First, we devise a concept encoder $\mathcal{E}$ that extracts concept representation c_{t-H} from the latest training sample $\mathcal{D}_{t-H} = (X_{t-H}, Y_{t-H})$. Let $X_{t-H}^{(i)} \in \mathbb{R}^L$ denote the time series of the i -th variate. Given all observations of N variates at time t , we encode c_{t-H} by the following equation:

$$c_{t-H} = \mathcal{E}(\mathcal{D}_{t-H}) = \text{Average} \left(\{ \text{MLP}(X_{t-H}^{(i)} \| Y_{t-H}^{(i)}) \}_{i=1}^N \right) \in \mathbb{R}^{d_c}, \quad (1)$$

where MLP extracts d_c latent features (e.g., mean and standard deviation) from each univariate time series, and Average yields the average latent features as a global concept that affects all variates.

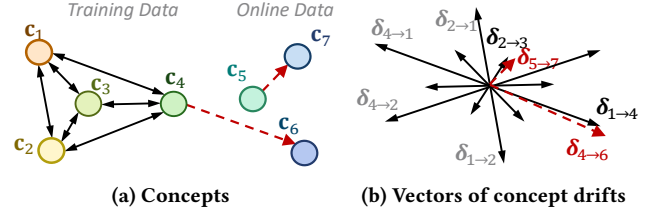


Figure 3: Example of synthetic concept drifts in historical training data and online concept drifts when $H = 2$. The circles represent concepts, while the arrows represent concept drifts.

Likewise, we employ another MLP to extract latent features from X_t , and the encoder $\mathcal{E}'$ extracts c_t by

$$c_t = \mathcal{E}'(X_t) = \text{Average} \left(\{ \text{MLP}'(X_t^{(i)}) \}_{i=1}^N \right) \in \mathbb{R}^{d_c}. \quad (2)$$

It is noteworthy that $\mathcal{E}'$ has the potential to estimate the hidden state of Y_t . When $L \geq kH$ and $k \geq 2$, X_t itself contains a sequence of horizon windows, i.e., $\{Y_{t-kH}, \dots, Y_{t-H}\}$. In this case, MLP' can learn the hidden state of each observed horizon window, exploit the temporal evolution pattern across horizon windows, and extrapolate the hidden state of the next horizon window Y_t .

Next, we estimate the concept drift between time $t-H$ and time t by the concept difference, i.e., $\delta_{t-H \rightarrow t} = c_t - c_{t-H} \in \mathbb{R}^{d_c}$.

4.3 Proactive Model Adaptation

As illustrated in Fig. 2, we assume the concept representation space and the parameter space to have some learnable relationships, where the estimated concept drift $\delta_{t-H \rightarrow t}$ can indicate the direction that the parameters θ_{t-H} should shift to.

Technically, it is non-trivial to decode the concept drift representation into an appropriate parameter shift. As the parameter space is often of a huge dimension, it is tough to search for an optimal mapping function between the concept space and the parameter space. Also, a simple mapping function may require too many additional parameters, leading to costly memory overhead. For instance, when $d_c = 100$ and the number of model parameters is 1 million, a trivial method is to learn a fully connected layer of 100 million parameters that maps $\delta_{t-H \rightarrow t}$ to parameter shifts $\Delta\theta$. To address these problems, as depicted in Fig. 4, we devise an adaptation generator $\mathcal{G}$ that employs bottleneck layers to produce a small number of adaptation coefficients $\alpha^{(\ell)}$ and $\beta^{(\ell)}$ for each the ℓ -th layer.

For simplicity, we take a linear transformation matrix as an example and denote the ℓ -th layer as $\theta_{t-H}^{(\ell)} \in \mathbb{R}^{d_{in} \times d_{out}}$. Given a small hyperparameter r ($r \ll \min(d_{in}, d_{out})$), we compute adaptation coefficients by

$$[\hat{\alpha}_t^{(\ell)}, \hat{\beta}_t^{(\ell)}] = W_2^{(\ell)\top} \left(\sigma \left(W_1^{(\ell)\top} \delta_{t-H \rightarrow t} + b^{(\ell)} \right) \right) + 1, \quad (3)$$

where $W_1^{(\ell)} \in \mathbb{R}^{r \times d_c}$, $b^{(\ell)} \in \mathbb{R}^r$, σ is the sigmoid activation, $W_2^{(\ell)} \in \mathbb{R}^{(d_{in}+d_{out}) \times r}$ is initialized by zeros, $\hat{\alpha}_t^{(\ell)} \in \mathbb{R}^{d_{in}}$, and $\hat{\beta}_t^{(\ell)} \in \mathbb{R}^{d_{out}}$.

To further reduce parameters, we share $W_1^{(\ell)}$ and $W_2^{(\ell)}$ across layers of the same type (e.g., up projection layers in all Transformer blocks), while we learn an individual bias term $b^{(\ell)}$ to customize

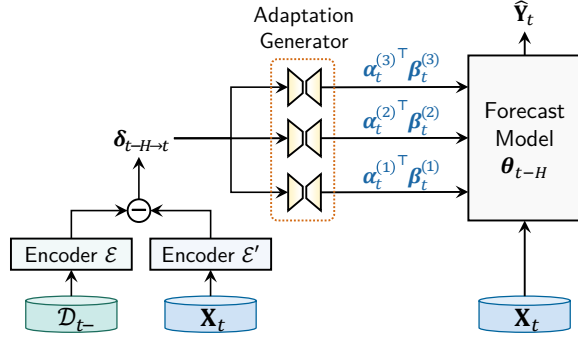


Figure 4: Overview of the model adapter in PROCEED that comprises two concept encoders and an adaptation generator.

distinct coefficients for each layer. Compared with a fully connected layer, we reduce the total parameters of the adaptation generator from $O(\mathcal{L}d_c d_{in} d_{out})$ to $O(r(\mathcal{L} + d_c + d_{in} + d_{out}))$, where $\mathcal{L}$ is the number of model layers.

Finally, we derive the adapted parameters used in online forecasting at time t by:

$$\widehat{\theta}_t^{(\ell)} = \left(\alpha_t^{(\ell)\top} \beta_t^{(\ell)} \right) \odot \theta_{t-H}^{(\ell)} \quad (4)$$

where $\odot$ is the element-wise product. In other words, we generate model adjustments $\Delta\theta^{(\ell)} = \left(\alpha_t^{(\ell)\top} \beta_t^{(\ell)} - 1 \right) \odot \theta_{t-H}^{(\ell)}$.

We also apply our method to convolution filters and bias terms. For a convolution filter $\theta^{(\ell)} \in \mathbb{R}^{d_{in} \times d_{out} \times d_k}$ with d_k kernels, we multiply each kernel with $\alpha_t^{(\ell)\top} \beta_t^{(\ell)}$. For a bias term $\theta^{(\ell)} \in \mathbb{R}^{d_{out}}$, we only generate $\beta_t^{(\ell)}$.

4.4 Mini-batch Training

Given abundant historical data, we shuffle them to synthesize diverse concept drifts and train our model adapter on them. To improve the training efficiency, we randomly select multiple samples as a mini-batch, adapt the forecaster towards each sample concurrently, and compute the average forecast loss.

Let $\mathcal{B}_k = \{\mathbf{X}_{k,j}, \mathbf{Y}_{k,j}\}_{j=kB+1}^{kB+B}$ represent the k -th mini-batch of B samples collected from different time. For the last mini-batch $\mathcal{B}_{k-1}$, our concept encoder $\mathcal{E}'$ extracts latent features of all samples and their average $\mathbf{c}_{k-1}$ is considered as the concept representation of $\mathcal{B}_{k-1}$. For each $\mathbf{X}_{k,j}$ in $\mathcal{B}_k$, we estimate the concept drift from $\mathcal{B}_{k-1}$ to $\mathbf{X}_{k,j}$ individually and generate the corresponding adaptation coefficients $\alpha_{k,j}^{(\ell)}, \beta_{k,j}^{(\ell)}$. Note that we do not iteratively make different versions of adapted models (i.e., $\{\widehat{\theta}_{k,j}\}_{j=kB+1}^{kB+B}$) which consumes more GPU memory and more time cost. Instead, we only preserve one model obtained from the last mini-batch (denoted as θ_{k-1}) and simultaneously handle multiple data batches.

For brevity, we omit the superscript (ℓ) in the following equations. Let $\mathbf{h}_{k,j} \in \mathbb{R}^{d_{in}}$ denote the intermediate inputs of the t -th

layer. Given adaptation coefficients $\alpha_{k,j}, \beta_{k,j}$, the linear transformation via adapted parameter $\widehat{\theta}_{k,j}$ is reformulated as

$$\begin{aligned} \widehat{\theta}_{k,j}^\top \mathbf{h}_{k,j} &= \left((\alpha_{k,j}^\top \beta_{k,j}) \odot \theta_{k-1} \right)^\top \mathbf{h}_{k,j} \\ &= \beta_{k,j} \odot \left(\theta_0^\top (\alpha_{k,j} \odot \mathbf{h}_{k,j}) \right). \end{aligned} \quad (5)$$

This equation also stands for other parameters such as convolution filters and bias terms. As such, we can directly compute $\theta_0^{(\ell)\top} \mathbf{H}'_k$ as usual, where $\mathbf{H}'_k = \{\alpha_{k,j} \odot \mathbf{h}_{k,j}\}_{j=kB+1}^{kB+B}$. Correspondingly, we multiply the layer outputs with $\beta_{k,j}$ for each sample in parallel.

As such, we obtain the predictions of all samples in $\mathcal{B}_k$, denoted as $\{\widehat{\mathbf{Y}}_{k,j}\}_{j=kB+1}^{kB+B}$. Next, we compute the average forecast loss on $\frac{1}{B} \sum_{j=kB+1}^{kB+B} \|\widehat{\mathbf{Y}}_{k,j} - \mathbf{Y}_{k,j}\|_2^2$ and use the gradients to update θ_{k-1} into θ_k and train other additional parameters, including the concept encoders and the adaptation generator. The updated model and the updated adapters will be used in the next mini-batch. During the online phase, we only fine-tune the forecast model and keep the parameters of the model adapter frozen because the ground truth of the test sample is not available.

As our model adapter learns the transition probability $P(\theta_t | \theta_s, \mathbf{X}_s, \mathbf{X}_t)$, we can follow the proof in the previous work [2] to show that it is feasible for $\widehat{\theta}_t$ in Eq. (4) to yield lower forecasting error than θ_{t-H} .

4.5 Discussion

4.5.1 Comparison with Existing Methods. Prior approaches to online time series forecasting perform model adaptation based on feedback in forecasting previous samples, which mainly focus on learning recent data patterns. In Table 2, we compare existing methods and PROCEED in terms of the training data and the model adaptation techniques at each online time t . Most methods use one newly acquired training sample, while SOLID [9] selects more recent samples that share similar lookback windows with the test sample and are assumed to share a similar concept. SOLID and PROCEED simply fine-tune the forecast model by gradient descent, while FSNet and OneNet further generate additional parameter adjustments based on the forecasting feedback. Since the feedback is delayed by H steps and cannot reveal the test concept, we propose a novel step called proactive model adaptation which aims to mitigate the effects of concept drift between the training sample and the test sample. As this core step is orthogonal to existing methods, PROCEED can incorporate the data augmentation technique of SOLID and the feedback-based model adaptation techniques of FSNet and OneNet.

4.5.2 Time Complexity Analysis. Though the lookback window size L and the number of variates N could be large, our concept drift estimation has a linear complexity w.r.t. them, i.e., $O(NLd_c)$ with a relatively small hyperparameter d_c . As for proactive model adaptation, the time complexity is $O(r(d_c + d_{in} + d_{out}) + d_{in}d_{out})$, which is agnostic to the number of variates. Note we set a rather small bottleneck dimension r (e.g., 32). Hence, our framework is friendly to large-scale multivariate time series with a large N . Throughout the online phase, we first adapt the model parameters by Eq. (4), and online forecasting is performed with no additional cost.

Table 2: Difference between online model adaptation methods.

Method	Training sample(s) $\mathcal{D}_{t-}$ at time t	Feedback-based model adaptation (How to adapt to $\mathcal{D}_{t-}$)	Proactive model adaptation (How to adapt to X_t)
FSNet [24]	$(X_{t-H}, Y_{t-H})^\dagger$	GD ‡ + Adjusting convolutional filters based on $\nabla \text{MSE}(\hat{Y}_{t-H}, Y_{t-H})$	$\times$
OneNet [32]	$(X_{t-H}, Y_{t-H})^\dagger$	GD ‡ + Adjusting ensemble weights based on $\text{MSE}(\hat{Y}_{t-H}, Y_{t-H})$	$\times$
SOLID [9]	$\{(X_s, Y_s) : s \leq t - H \wedge \ X_s - X_t\ _2 \in \text{Top}K\}$	GD ‡	$\times$
 PROCEED	(X_{t-H}, Y_{t-H})	GD ‡	Adjusting all parameters based on X_{t-H} and X_t

† FSNet and OneNet used (X_{t-1}, Y_{t-1}) in their official implementations, while the practical strategy is to use (X_{t-H}, Y_{t-H}) without information leakage.

‡ "GD" refers to applying gradient descent algorithm on $\mathcal{D}_{t-}$.

Table 3: MSE of different online methods with horizons in {24, 48, 96}. The best results are marked in bold.

Model	Dataset Method H	ETTh2			ETTm1			Weather			ECL			Traffic		
		24	48	96	24	48	96	24	48	96	24	48	96	24	48	96
TCN	\	3.156	4.141	6.019	1.770	1.908	1.673	3.301	9.718	8.588	57.782	57.948	58.729	0.569	0.637	0.676
	GD	3.223	4.408	6.291	1.015	1.172	1.142	0.868	1.346	1.799	6.315	8.176	11.385	0.452	0.509	0.576
	FSNet	3.079	4.247	6.213	0.763	0.923	1.003	0.875	1.338	1.755	6.143	9.435	12.941	0.458	0.517	0.574
	OneNet	2.965	4.892	8.257	1.335	1.547	1.040	0.861	1.182	1.484	9.757	10.847	15.932	0.462	0.544	0.591
	SOLID++	3.231	4.111	6.191	0.674	0.795	0.868	0.867	1.311	1.670	5.991	7.340	9.412	0.418	0.479	0.529
	PROCEED	2.908	4.056	5.891	0.531	0.704	0.780	0.707	0.959	1.314	5.907	7.192	9.183	0.413	0.454	0.511
PatchTST	\	2.880	4.103	6.178	0.549	0.726	0.769	0.746	1.006	1.311	4.207	4.942	6.042	0.378	0.386	0.403
	GD	2.092	3.434	5.770	0.455	0.598	0.674	0.735	0.979	1.261	4.143	4.762	5.791	0.376	0.380	0.398
	OneNet	2.717	4.095	6.044	0.681	0.721	0.788	0.824	1.052	1.319	4.111	4.752	5.767	0.360	0.372	0.387
	SOLID++	2.648	3.925	6.148	0.447	0.580	0.659	0.735	0.981	1.262	4.156	4.780	5.835	0.376	0.378	0.397
	PROCEED	1.735	3.114	5.555	0.424	0.577	0.660	0.724	0.973	1.261	3.958	4.604	5.635	0.335	0.357	0.376
iTransformer	\	3.605	4.992	7.114	0.603	0.807	0.832	0.942	1.211	1.461	4.082	4.874	6.054	0.354	0.372	0.393
	GD	2.637	4.148	6.734	0.468	0.606	0.695	0.869	1.143	1.409	4.055	4.851	5.989	0.348	0.369	0.388
	OneNet	2.985	4.015	6.451	0.609	0.746	0.766	0.803	1.089	1.418	4.077	4.906	6.100	0.349	0.371	0.395
	SOLID++	2.804	4.278	6.582	0.455	0.601	0.688	0.875	1.139	1.403	4.070	4.816	6.024	0.342	0.365	0.384
	PROCEED	2.387	3.969	6.291	0.426	0.561	0.642	0.742	1.015	1.294	3.899	4.647	5.710	0.330	0.353	0.373

Table 4: MAE of different online methods with horizons in {24, 48, 96}. The best results are marked in bold.

Model	Dataset Method H	ETTh2			ETTm1			Weather			ECL			Traffic		
		24	48	96	24	48	96	24	48	96	24	48	96	24	48	96
TCN	\	0.729	0.800	0.921	0.771	0.819	0.782	0.819	1.909	1.453	0.659	0.676	0.693	0.351	0.392	0.404
	GD	0.727	0.824	0.946	0.635	0.674	0.667	0.406	0.561	0.703	0.401	0.446	0.488	0.320	0.350	0.383
	FSNet	0.705	0.829	0.957	0.521	0.581	0.609	0.406	0.558	0.690	0.390	0.436	0.474	0.325	0.355	0.384
	OneNet	0.728	0.902	1.096	0.688	0.781	0.678	0.456	0.591	0.707	0.767	0.721	0.505	0.319	0.366	0.393
	SOLID++	0.711	0.799	0.916	0.524	0.566	0.590	0.398	0.543	0.667	0.394	0.430	0.472	0.308	0.334	0.353
	PROCEED	0.659	0.767	0.890	0.447	0.521	0.553	0.382	0.493	0.637	0.387	0.431	0.463	0.291	0.308	0.332
PatchTST	\	0.684	0.770	0.891	0.450	0.528	0.550	0.373	0.491	0.612	0.297	0.321	0.346	0.277	0.276	0.286
	GD	0.611	0.718	0.864	0.407	0.474	0.509	0.369	0.482	0.592	0.294	0.315	0.341	0.276	0.267	0.278
	OneNet	0.631	0.731	0.862	0.477	0.504	0.541	0.392	0.496	0.605	0.291	0.314	0.340	0.259	0.259	0.263
	SOLID++	0.670	0.761	0.890	0.406	0.467	0.504	0.372	0.485	0.597	0.294	0.315	0.340	0.276	0.264	0.277
	PROCEED	0.579	0.692	0.849	0.392	0.463	0.505	0.367	0.477	0.591	0.283	0.307	0.339	0.233	0.244	0.249
iTransformer	\	0.724	0.822	0.944	0.478	0.566	0.584	0.473	0.581	0.676	0.289	0.315	0.344	0.257	0.267	0.280
	GD	0.669	0.788	0.943	0.422	0.488	0.529	0.444	0.552	0.651	0.287	0.313	0.341	0.251	0.262	0.272
	OneNet	0.663	0.775	0.916	0.449	0.525	0.546	0.411	0.526	0.633	0.289	0.316	0.345	0.252	0.264	0.277
	SOLID++	0.673	0.789	0.929	0.421	0.488	0.527	0.447	0.554	0.653	0.288	0.314	0.343	0.248	0.260	0.269
	PROCEED	0.633	0.753	0.889	0.398	0.461	0.500	0.378	0.495	0.602	0.281	0.306	0.331	0.231	0.243	0.254

5 Experiments

In this section, we present experiments on real-world datasets to evaluate the effectiveness and efficiency of PROCEED.

5.1 Experimental Settings

Datasets. As introduced in Sec. 3.1, we use five popular benchmarks, of which more details are provided in Appendix B.1. We

conform to the dataset split ratio of FSNet and OneNet, *i.e.*, we split the datasets by 20:5:75 for training, validation, and testing. The rationale is that online learning is of great practical value in scenarios of limited training data, when pretrained models are inadequate at handling new concepts during long-term online service.

Forecast Models. As our framework is model-agnostic, we pre-train three popular and advanced forecasting models, including TCN [3], PatchTST [23], and iTransformer [19]. We report the forecasting errors of these pretrained models without any online learning method (Method=" " in Table 3-4).

Online Learning Baselines. We compare PROCEED with GD, a naïve online gradient descent method, and the state-of-the-art online model adaptation methods, including FSNet, OneNet, and SOLID. We reimplement FSNet and OneNet by adopting the *Practical* strategy described in Sec. 3.2. Furthermore, FSNet is specially designed for TCN, while OneNet is based on online ensembling and is model-agnostic. We implement multiple variants of OneNet by combining each forecast model with an FSNet. As for SOLID, its original work learns linear probing with the pretrained parameters at each update time and does not inherit the fine-tuned parameters from the last update. As our datasets have a much longer online phase, we implement a variant called SOLID++ that continually fine-tunes all model parameters across the online data. Empirically, SOLID++ performs better than SOLID in our evaluation setting.

5.2 Overall Comparison

5.2.1 Effectiveness. To verify the effectiveness of our proposed framework, we compare the predictive performance of PROCEED and other baselines on the five popular datasets. We repeat each experiment with 3 runs and report the average results. As shown in Table 3-4, PROCEED achieves the best performance in most cases, reducing the average forecast errors of all models without online learning by 42.3%, 10.3%, 12.9% for TCN, PatchTST, and iTransformer, respectively. Also, it is worth mentioning that all forecast models have been enhanced by RevIN [16], a data adaptation approach to concept drift by reducing the distribution shifts w.r.t. the mean and standard deviation of time series. Since all online learning methods can still achieve remarkable improvements, it suffices to support our claim that time series forecast models need model adaptation to handle more complex concept drifts.

Based on the same forecast model, PROCEED outperforms the existing online model adaptation methods by 12.5%, 13.6%, 6.7% against FSNet, OneNet, and SOLID++, respectively. In particular, we can observe that TCN with existing online learning methods still lags behind a frozen PatchTST. By contrast, TCN enhanced by PROCEED can outperform the frozen PatchTST and iTransformer in some cases of ETTh2, ETTm1, and Weather datasets. Note that these 3 datasets have relatively more significant concept drift as shown in Table 1. This indicates that PROCEED has the capability of handling the concept drift during online learning.

5.2.2 Efficiency. As depicted in Fig. 5, we compare the GPU memory occupation and inference latency of different model adaptation methods on Traffic, the largest time series dataset. We record the online latency, including feedback-based model adaptation and any other additional steps. Here, SOLID only fine-tunes the final layer

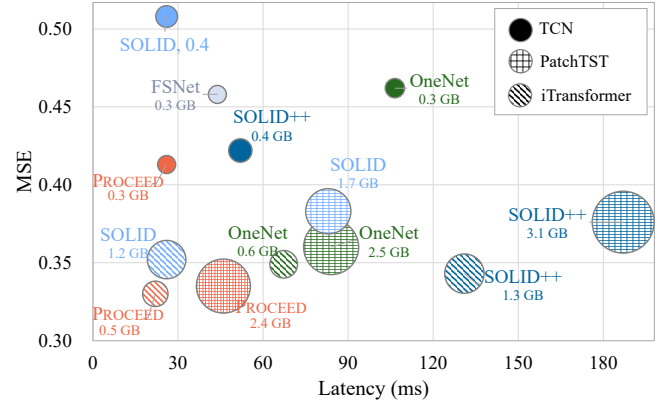


Figure 5: Efficiency comparison on the Traffic dataset ($H = 24$). The horizontal axis is the online forecasting latency (millisecond) between updating the model and obtaining online predictions. The vertical axis is the average MSE on test data. The size of each circle represents the peak amount of GPU memory occupation (GB).

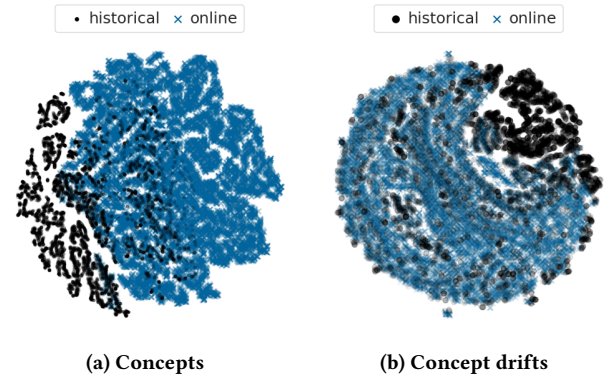


Figure 6: Visualization of concept representations and concept drift representations on ETTm1 ($H = 24$).

rather than the whole model, reducing the gradient computation cost. More specifically, SOLID and SOLID++ compute gradients on several selected training samples iteratively instead of concurrently, so as to save GPU memory. By contrast, PROCEED only uses the latest training sample and thereby achieves the lowest latency. Moreover, PROCEED is faster and more lightweight than the online ensembling method OneNet.

5.3 Visualization of the Representation Space

To verify our assumption that online data have OOD concepts, we adopt t-SNE to visualize the representations of concepts c_t and concept drifts $\delta_{t-H \rightarrow t}$ on the historical training data and online data. As shown in Fig. 6a, there are a great number of OOD concepts in the online data, which are distinct from any historical concept. By contrast, as shown in Fig. 6b, we encounter much fewer OOD concept drifts on the online data. These results support our intuition illustrated in Fig. 3. Hence, it is desirable for the adaptation

Table 5: Ablation study results. We report the average MSE with horizons in $\{24, 48, 96\}$ and the relative MSE increase ($\bar{\Delta}_{\text{MSE}}$) of the variants against PROCEED on each dataset.

Model	Method	ETTh2	ETTm1	Weather	ECL	Traffic	$\bar{\Delta}_{\text{MSE}}$
PatchTST	feedback-only	3.765	0.576	0.992	4.899	0.385	4.9%
	$\mathcal{G}(\mathbf{c}_t)$	4.194	0.607	0.997	4.742	0.358	6.5%
	$\mathcal{E}'(\mathbf{X}_{t-H})$	3.702	0.678	1.011	4.974	0.363	7.8%
	diff. $\mathbf{W}_1^{(\ell)}, \mathbf{W}_2^{(\ell)}$	3.573	0.578	0.993	4.765	0.358	1.9%
	PROCEED	3.468	0.554	0.986	4.732	0.356	-
iTransformer	feedback-only	4.506	0.590	1.140	4.965	0.368	7.3%
	$\mathcal{G}(\mathbf{c}_t)$	4.218	0.558	1.030	4.842	0.352	1.2%
	$\mathcal{E}'(\mathbf{X}_{t-H})$	4.396	0.557	1.030	4.776	0.352	1.7%
	diff. $\mathbf{W}_1^{(\ell)}, \mathbf{W}_2^{(\ell)}$	4.254	0.547	1.020	4.767	0.352	0.4%
	PROCEED	4.216	0.543	1.017	4.752	0.352	-

generator to generate adaptations based on concept drifts instead of concepts.

5.4 Ablation Study

For a fine-grained investigation into the role of our proposed components, we conduct more ablation studies by introducing five variants as follows. (1) *feedback-only*: this variant only performs gradient descent based on feedback from $\mathcal{D}_{t-}$; (2) $\mathcal{G}(\mathbf{c}_t)$: this variant generates adaptation based on the concept of the test sample only instead of estimating concept drift; (3) $\mathcal{E}'(\mathbf{X}_{t-H})$: this variant uses the same encoder $\mathcal{E}'$ to extract concepts $\mathbf{c}_{t-H}$ and $\mathbf{c}_t$ from look-back windows $\mathbf{X}_{t-H}$ and $\mathbf{X}_t$, respectively; (4) *diff. $\mathbf{W}_1^{(\ell)}, \mathbf{W}_2^{(\ell)}$* : the bottleneck layers that generate adaptation coefficients are totally different across the model layers.

As shown in Table 5, we have four major observations. First, PROCEED outperforms the naïve feedback-based model adaptation method by a large margin, demonstrating the necessity of proactive model adaptation in reducing the distribution gap between new training data and test data. Second, the variant w/ $\mathcal{G}(\mathbf{c}_t)$, which generates adaptation only based on the estimated test concept, results in a considerably higher MSE than PROCEED based on concept drift. This is due to the fact that some OOD concepts may take place in the online phase and challenge the adaptation generator. By contrast, our concept drift-based design can make the inputs of $\mathcal{G}$ in-distribution as much as possible, reducing difficulties in generating adaptation. Third, the variant w/ $\mathcal{E}'(\mathbf{X}_{t-H})$ has suboptimal performance compared with PROCEED with two different concept encoders. One reason is that we can only infer limited information about $\mathbf{Y}_{t-H}$ from $\mathbf{X}_{t-H}$, and it would be better to leverage the observed $\mathbf{Y}_{t-H}$. Another possible reason is that the model before proactive model adaptation does not overfit $\mathcal{D}_{t-}$, while we anticipate the adapted model being optimal for $(\mathbf{X}_t, \mathbf{Y}_t)$. Thus, $\mathcal{E}$ should encode a little information of the latest training sample into $\mathbf{c}_{t-H}$, while a different encoder $\mathcal{E}'$ should encode all information of the test sample into $\mathbf{c}_t$. Fourth, the variant learning different $\mathbf{W}_1^{(\ell)}$ and $\mathbf{W}_2^{(\ell)}$ do not significantly outperform PROCEED with weights shared across some layers. Considering the non-stationarity of time series, an over-parameterized generator $\mathcal{G}$ has higher overfitting risks on training data. Thus we only set the bias term $\mathbf{b}_1^{(\ell)}$ to be different across model layers, yielding distinct, layer-specific adaptations.

Table 6: MSE of PROCEED with various concept encoders.

Model	Method	ETTh2	ETTm1	Weather	ECL	Traffic
PatchTST	linear trans.	3.498	0.570	0.985	5.661	0.363
	weighted sum	3.467	0.571	0.986	5.435	0.362
	average	3.468	0.554	0.986	4.732	0.356
iTransfo.	linear trans.	4.400	0.548	1.018	4.944	0.355
	weighted sum	4.441	0.551	1.021	5.260	0.352
	average	4.216	0.543	1.017	4.752	0.352

Besides, we study different kinds of operations on multivariate time series using our concept encoder. Let $\mathbf{c}_t^{(i)}$ denote the latent concept feature vector of the i -th variate. Apart from the average introduced in Eq. (1), we implement two variants as follows:

- **linear transformation**: we learn a linear layer that transforms the concatenation of all latent features, i.e., $\mathbf{c}_t = W^T [\mathbf{c}_t^{(0)}, \mathbf{c}_t^{(1)}, \dots]$.
- **weighted sum**: we learn an individual weight $w_i \in \mathbb{R}$ for each i -th variate, and $\mathbf{c}_t = \sum_i w_i \mathbf{c}_t^{(i)}$.

In Table 6, we compare the performance of the variants in terms of the average MSE with $H \in \{24, 48, 96\}$. The results demonstrate that the average operation is simple yet effective. One possible reason is that all variates are typically assumed to have equal importance during both training and evaluation.

6 Conclusion

In this work, we highlight that online time series forecasting inherently has a temporal gap between each test sample and available training data, where concept drift may well occur. Through empirical study, we found that this gap hinders the effectiveness of existing model adaptation methods which passively rely on the feedback in forecasting recent data. To address this issue, we propose a novel online model adaptation framework named PROCEED, which proactively adapts the forecast model to the test concept before forecasting each test sample. Specifically, PROCEED first fine-tunes the model on the latest acquired training sample, then extracts latent features from time series data to estimate the undergoing concept drift, and efficiently maps the estimated drift into parameter adjustments that are tailored for the test sample. Furthermore, we synthesize diverse concept drift and optimize PROCEED with the generalization capacity of mapping concept drift to beneficial parameter adjustments. Extensive experiments on five real-world time series datasets demonstrate that our proposed PROCEED remarkably reduces the forecast errors of various forecast models and surpasses the state-of-the-art online learning methods. Our proactive model adaptation method provides a new direction in addressing continuous concept drift for online systems, which we wish to help any other prediction tasks with a feedback delay issue.

Acknowledgments

This work is supported by the National Key Research and Development Program of China (2022YFE0200500), Shanghai Municipal Science and Technology Major Project (2021SHZDZX0102), and SJTU Global Strategic Partnership Fund (2021 SJTU-HKUST). We would like to thank Shaofeng Cai for his valuable advice on the preliminary version of the paper.

References

- [1] Anonymous. 2024. Fast and Slow Streams for Online Time Series Forecasting Without Information Leakage. In *Submitted to The Thirteenth International Conference on Learning Representations*. <https://openreview.net/forum?id=10n3EyogMi> under review.
- [2] Guangji Bai, Chen Ling, and Liang Zhao. 2023. Temporal Domain Generalization with Drift-Aware Dynamic Neural Networks. In *The Eleventh International Conference on Learning Representations*. <https://openreview.net/forum?id=sW0sRj4nT1n>
- [3] Shaojie Bai, J. Zico Kolter, and Vladlen Koltun. 2018. An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling. (March 2018). arXiv:1803.01271 [cs.LG]
- [4] Kaifeng Bi, Lingxi Xie, Hengheng Zhang, Xin Chen, Xiaotao Gu, and Qi Tian. 2023. Accurate medium-range global weather forecasting with 3D neural networks. *Nature* 619, 7970 (2023), 533–538. doi:10.1038/s41586-023-06185-3
- [5] Oussama Boussif, Ghait Boukachab, Dan Assouline, Stefano Massaroli, Tianle Yuan, Loubna Benabbou, and Yoshua Bengio. 2023. Improving day-ahead Solar Irradiance Time Series Forecasting by Leveraging Spatio-Temporal Context. In *Advances in Neural Information Processing Systems*, Vol. 36. Curran Associates, Inc., 2342–2367.
- [6] Pietro Buzzega, Matteo Boschini, Angelo Porrello, Davide Abati, and SIMONE CALDERARA. 2020. Dark Experience for General Continual Learning: a Strong, Simple Baseline. In *Advances in Neural Information Processing Systems*, Vol. 33. Curran Associates, Inc., 15920–15930.
- [7] Joos-Hendrik Böse, Valentin Flunkert, Jan Gasthaus, Tim Januschowski, Dustin Lange, David Salinas, Sebastian Schelter, Matthias Seeger, and Yuyang Wang. 2017. Probabilistic demand forecasting at scale. *Proceedings of the VLDB Endowment* 10, 12 (Aug. 2017), 1694–1705. doi:10.14778/3137765.3137775
- [8] Massimo Caccia, Pau Rodríguez, Oleksiy Ostapenko, Fabrice Normandin, Min Lin, Lucas Page-Caccia, Issam Hadj Laradji, Irina Rish, Alexandre Lacoste, David Vázquez, and Laurent Charlin. 2020. Online Fast Adaptation and Knowledge Accumulation (OSAKA): a New Approach to Continual Learning. In *NeurIPS*.
- [9] Mouxiang Chen, Lefei Shen, Han Fu, Zhuo Li, Jianling Sun, and Chenghao Liu. 2024. Calibration of Time-Series Forecasting: Detecting and Adapting Context-Driven Distribution Shift. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (Barcelona, Spain) (KDD '24)*. Association for Computing Machinery, New York, NY, USA, 341–352. doi:10.1145/3637528.3671926
- [10] Yuntao Du, Jindong Wang, Wenjie Feng, Sinno Pan, Tao Qin, Renjun Xu, and Chongjun Wang. 2021. AdaRNN: Adaptive Learning and Forecasting for Time Series. In *Proceedings of the 30th ACM International Conference on Information & Knowledge Management*. ACM. doi:10.1145/3459637.3482315
- [11] Vijay Ekambaram, Arindam Jati, Nam Nguyen, Phanwadee Sinthong, and Jayant Kalagnanam. 2023. TSMixer: Lightweight MLP-Mixer Model for Multivariate Time Series Forecasting. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (Long Beach, CA, USA) (KDD '23)*. Association for Computing Machinery, New York, NY, USA, 459–469. doi:10.1145/3580305.3599533
- [12] Wei Fan, Pengyang Wang, Dongkun Wang, Dongjie Wang, Yuanchun Zhou, and Yanjie Fu. 2023. Dish-TS: A General Paradigm for Alleviating Distribution Shift in Time Series Forecasting. In *AAAI Conference on Artificial Intelligence*. <https://api.semanticscholar.org/CorpusID:257232506>
- [13] João Gama, Indrè Žliobaitė, Albert Bifet, Mykola Pechenizkiy, and Abdelhamid Bouchachia. 2014. A Survey on Concept Drift Adaptation. *Comput. Surveys* 46, 4 (apr 2014), 1–37. doi:10.1145/2523813
- [14] Lu Han, Xu-Yang Chen, Han-Jia Ye, and De-Chuan Zhan. 2024. SOFTS: Efficient Multivariate Time Series Forecasting with Series-Core Fusion. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*. <https://openreview.net/forum?id=89AUi5L1uA>
- [15] Steven C.H. Hoi, Doyen Sahoo, Jing Lu, and Peilin Zhao. 2021. Online learning: A comprehensive survey. *Neurocomputing* 459 (Oct. 2021), 249–289. doi:10.1016/j.neucom.2021.04.112
- [16] Taesung Kim, Jinhee Kim, Yunwon Tae, Cheonbok Park, Jang-Ho Choi, and Jaegul Choo. 2022. Reversible Instance Normalization for Accurate Time-Series Forecasting against Distribution Shift. In *International Conference on Learning Representations*. <https://openreview.net/forum?id=cGDakQo1C0p>
- [17] Wendi Li, Xiao Yang, Weiqing Liu, Yingce Xia, and Jiang Bian. 2022. DDG-DA: Data Distribution Generation for Predictable Concept Drift Adaptation. *Proceedings of the AAAI Conference on Artificial Intelligence* 36, 4 (jun 2022), 4092–4100. doi:10.1609/aaai.v36i4.20327
- [18] Daojun Liang, Haixia Zhang, Jing Wang, Dongfeng Yuan, and Minggao Zhang. 2024. Act Now: A Novel Online Forecasting Framework for Large-Scale Streaming Data. arXiv:2412.00108 [cs.LG] <https://arxiv.org/abs/2412.00108>
- [19] Yong Liu, Tengge Hu, Haoran Zhang, Haixu Wu, Shiyu Wang, Lintao Ma, and Mingsheng Long. 2024. iTransformer: Inverted Transformers Are Effective for Time Series Forecasting. In *The Twelfth International Conference on Learning Representations*. <https://openreview.net/forum?id=jEPfA18fah>
- [20] Yong Liu, Haixu Wu, Jianmin Wang, and Mingsheng Long. 2022. Non-stationary Transformers: Exploring the Stationarity in Time Series Forecasting. In *Advances in Neural Information Processing Systems*, Vol. 35. 9881–9893.
- [21] Zhiding Liu, Mingyue Cheng, Zhi Li, Zhenya Huang, Qi Liu, Yanhu Xie, and Enhong Chen. 2023. Adaptive Normalization for Non-stationary Time Series Forecasting: A Temporal Slice Perspective. In *Thirty-seventh Conference on Neural Information Processing Systems*. <https://openreview.net/forum?id=5BqDSw8r5j>
- [22] Michael McCloskey and Neal J. Cohen. 1989. Catastrophic Interference in Connectionist Networks: The Sequential Learning Problem. In *Psychology of Learning and Motivation*. Elsevier, 109–165. doi:10.1016/s0079-7421(08)60536-8
- [23] Yuqi Nie, Nam H Nguyen, Phanwadee Sinthong, and Jayant Kalagnanam. 2023. A Time Series is Worth 64 Words: Long-term Forecasting with Transformers. In *The Eleventh International Conference on Learning Representations*. <https://openreview.net/forum?id=Jbdc0vTOcol>
- [24] Quang Pham, Chenghao Liu, Doyen Sahoo, and Steven Hoi. 2023. Learning Fast and Slow for Online Time Series Forecasting. In *The Eleventh International Conference on Learning Representations*. <https://openreview.net/forum?id=q-PbpHD3EOk>
- [25] Jonathan Schwarz, Wojciech Czarnecki, Jelen Luketina, Agnieszka Grabska-Barwinska, Yee Whye Teh, Razvan Pascanu, and Raia Hadsell. 2018. Progress & Compress: A scalable framework for continual learning. In *Proceedings of the 35th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 80)*. PMLR, 4528–4537.
- [26] Chenchen Sun, Yan Ning, Derong Shen, and Tiezheng Nie. 2023. Graph Neural Network-Based Short-Term Load Forecasting with Temporal Convolution. *Data Science and Engineering* 9, 2 (Nov. 2023), 113–132. doi:10.1007/s41019-023-00233-8
- [27] Liyan Wang, Xingxing Zhang, Hang Su, and Jun Zhu. 2023. A Comprehensive Survey of Continual Learning: Theory, Method and Application. (Jan. 2023). arXiv:2302.00487 [cs.LG]
- [28] Gerald Woo, Chenghao Liu, Akshat Kumar, Caiming Xiong, Silvio Savarese, and Doyen Sahoo. 2024. Unified Training of Universal Time Series Forecasting Transformers. In *Proceedings of the 41st International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 235)*. PMLR, 53140–53164.
- [29] Zonghan Wu, Shirui Pan, Guodong Long, Jing Jiang, Xiaojun Chang, and Chengqi Zhang. 2020. Connecting the Dots: Multivariate Time Series Forecasting with Graph Neural Networks. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. ACM. doi:10.1145/3394486.3403118
- [30] Xiaoyu You, Mi Zhang, Daizong Ding, Fuli Feng, and Yuanmin Huang. 2021. Learning to Learn the Future: Modeling Concept Drift in Time Series Prediction. In *Proceedings of the 30th ACM International Conference on Information & Knowledge Management*. ACM. doi:10.1145/3459637.3482271
- [31] Ailing Zeng, Muxi Chen, Lei Zhang, and Qiang Xu. 2023. Are Transformers Effective for Time Series Forecasting? *Proceedings of the AAAI Conference on Artificial Intelligence* 37, 9 (jun 2023), 11121–11128. doi:10.1609/aaai.v37i9.26317
- [32] YiFan Zhang, Qingsong Wen, Xue Wang, Weiqi Chen, Liang Sun, Zhang Zhang, Liang Wang, Rong Jin, and Tieniu Tan. 2023. OneNet: Enhancing Time Series Forecasting Models under Concept Drift by Online Ensembling. In *Thirty-seventh Conference on Neural Information Processing Systems*. <https://openreview.net/forum?id=Q25wMXsacZ>
- [33] Lifan Zhao, Shuming Kong, and Yanyan Shen. 2023. DoubleAdapt: A Meta-learning Approach to Incremental Learning for Stock Trend Forecasting. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (Long Beach, CA, USA) (KDD '23)*. Association for Computing Machinery, New York, NY, USA, 3492–3503. doi:10.1145/3580305.3599315
- [34] Lifan Zhao and Yanyan Shen. 2024. Rethinking Channel Dependence for Multivariate Time Series Forecasting: Learning from Leading Indicators. In *The Twelfth International Conference on Learning Representations*. <https://openreview.net/forum?id=JiTvtCUOpS>
- [35] Haoyi Zhou, Shanghang Zhang, Jieqi Peng, Shuai Zhang, Jianxin Li, Hui Xiong, and Wancai Zhang. 2021. Informer: Beyond Efficient Transformer for Long Sequence Time-Series Forecasting. *Proceedings of the AAAI Conference on Artificial Intelligence* 35, 12 (may 2021), 11106–11115. doi:10.1609/aaai.v35i12.17325
- [36] Tian Zhou, Peisong Niu, Xue Wang, Liang Sun, and Rong Jin. 2023. One Fits All: Power General Time Series Analysis by Pretrained LM. In *Thirty-seventh Conference on Neural Information Processing Systems*. <https://openreview.net/forum?id=gMS6fVZvmF>
- [37] Jiaqi Zhu, Shaofeng Cai, Fang Deng, Beng Chin Ooi, and Wenqiao Zhang. 2023. METER: A Dynamic Concept Adaptation Framework for Online Anomaly Detection. *Proceedings of the VLDB Endowment* 17, 4 (Dec. 2023), 794–807. doi:10.14778/3636218.3636233

Table 7: The statistics of five popular time series datasets.

Dataset	# Sensors (N)	# Time Steps	Frequency
ETTh2	7	17,420	1 hour
ETTm1	7	69,680	15 min
Weather	21	52,696	10 min
ECL	321	26,304	1 hour
Traffic	862	17,544	1 hour

A Related Works

A.1 Online Model Adaptation

Online model adaptation, or online learning [15], has been a popular learning paradigm that updates models on new data instantly or periodically. In the general field, most efforts focus on addressing the catastrophic forgetting issue [22] stemming from excessive updates, and numerous *continual learning* [27] methods have been developed to retain acquired knowledge of past data by rehearsal mechanisms [6] and regularization terms [25], which can be seamlessly incorporated into our framework. Most recently, SOLID [9] proposed to adapt the forecast model by several selected training samples which are assumed to have a similar context (*i.e.*, concept in this paper) to each test sample. SOLID [9] relies on heuristic measures about context similarity and is not fully aware of unobserved contexts. The selected training samples may not share the exactly same concept with the test sample. Thus it is desirable to further adopt our proposed framework. We would like to leave the combination of training data sampling and proactive model adaptation as future work.

After the submission of our manuscript, there are two concurrent researches [1, 18] that also noticed the information leakage in FSNet and OneNet. Since the ground-truth H -step values of the last prediction are not fully observed, both works propose to generate pseudo labels of the unobserved values, concatenate the pseudo labels with observed labels to simulate complete ground truth, and calculate the forecast errors to update the model. The potential drawback is that the pseudo label generator is still susceptible to concept drift and may produce low-quality pseudo labels, leaving the concept drift issue unresolved.

On top of online learning, *rolling retraining* [17] is another learning paradigm that periodically re-trains a new model from scratch on all historical data, while the cost of frequent retraining is unaffordable. In practice, we can perform rolling retraining once a month and adopt daily updates during the interval to learn new patterns timely. Our work is orthogonal to rolling retraining.

A.2 Data Adaptation

Apart from model adaptation, data adaptation is another mainstream approach to concept drift in time series forecasting, which is orthogonal to model adaptation. The goal of data adaptation is to normalize historical training data and future test data into a common data distribution [12, 16, 21], reducing overfitting risks. Among them, RevIN [16] is the most popular method that applies instance normalization to each time series lookback window and restores the statistics to the corresponding predictions, making each sample follow a similar distribution. Such normalization-based data adaptation methods mainly focus on the statistical changes in the mean

and deviation of time series, while they overlook the distribution shifts in more complex temporal dependencies between time steps and spatial dependencies between variates [34]. Meanwhile, the statistics may also serve as informative signals for prediction [20].

B Experimental Setup

B.1 Datasets

We list descriptions of datasets used for experiments as follows.

- ETT [35] (Electricity Transformer Temperature) records seven features of electricity transformers from 2016 to 2018. ETTh1 is hourly collected and ETTm1 is 15-minutely collected.
- ECL [31] records the hourly electricity consumption of 321 clients from 2012 to 2014.
- Weather [35] records 21 features of weather, *e.g.*, air temperature and humidity in 2020.
- Traffic [29] records the hourly road occupancy rates recorded by the sensors of San Francisco freeways from 2015 to 2016.

B.2 Evaluation Settings

To the best of our knowledge, there is no open-source evaluation framework that considers the temporal gap in online learning for time series forecasting. Though Zhang *et al.* [32] noticed the feedback delay, they sidestep this issue by performing one prediction task every H time steps and performing the next task until the ground truth is available. Concretely, they use $\mathbf{X}_{t-L:t}$ to forecast $\mathbf{X}_{t+1:t+H}$ and then use $\mathbf{X}_{t-L+H:t+H}$ to forecast $\mathbf{X}_{t+H+1:t+2H}$. In contrast, the common practice is to perform forecasting at each time step between t and $t+H$, continually updating the previous predictions of $\mathbf{X}_{t-L+H:t+H}$. The reason is that short-term forecasting is typically easier than long-term forecasting. Therefore, in this work, we implement a more realistic evaluation framework for online learning where we perform forecasting at each time step, which remains consistent with the traditional evaluation setting.

As for the ETTm1 dataset, popular works (*e.g.*, Informer [35], PatchTST [23], and iTransformer [19]) usually use 69,680 time steps of the dataset. In contrast, FSNet and OneNet only used a quarter of it, *i.e.*, 17,420 steps, and the training set only contains 3484 samples. This setting is in favor of small models (*e.g.*, TCN) but is challenging for the state-of-the-art time series forecast models. Therefore, we take all 69,680 time steps following PatchTST and iTransformer. As such, the ETTm1 dataset in our experiment has the greatest number of samples, enriching the diversity of our datasets.

B.3 Implementation Details

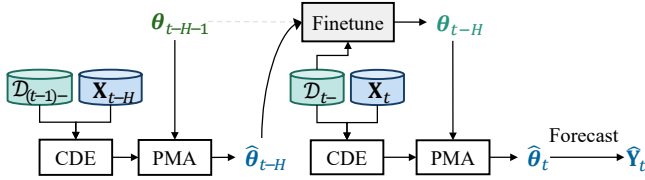
We conduct experiments on four Nvidia 4090 24GB GPUs. We ran each experiment 3 times with different random seeds and reported the average results. We follow the official implementation of the forecast models and the online learning methods, using their recommended model hyperparameters (*e.g.*, the number of layers) and the Adam optimizer. In cases where default hyperparameter values are not provided, we conduct a grid search to find the optimal hyperparameters that yield the best performance. Following FSNet and OneNet, the lookback length L for TCN is set to be 60. For PatchTST and iTransformer, we search for the optimal lookback length and set L to be 512. We enhance TCN and FSNet by RevIN [16] which

Table 8: MAE results of two online learning strategies for time series forecasting. The horizon H varies in 24, 48, and 96.

Model	Dataset H	ETTh2			ETTm1			Weather			ECL			Traffic		
		24	48	96	24	48	96	24	48	96	24	48	96	24	48	96
FSNet	Optimal	0.467	0.515	0.596	0.472	0.538	0.539	0.396	0.471	0.511	0.397	0.434	0.452	0.318	0.348	0.370
	Practical	0.705	0.829	0.957	0.521	0.581	0.609	0.406	0.558	0.690	0.390	0.436	0.474	0.325	0.355	0.384
	Δ_{MAE}	51%	61%	61%	10%	8%	13%	2%	19%	35%	-2%	0%	5%	2%	2%	4%
OneNet	Optimal	0.407	0.436	0.545	0.272	0.275	0.286	0.238	0.261	0.276	0.333	0.348	0.387	0.300	0.320	0.340
	Practical	0.728	0.902	1.096	0.688	0.781	0.678	0.456	0.591	0.707	0.767	0.721	0.505	0.319	0.366	0.393
	Δ_{MAE}	79%	107%	101%	153%	184%	137%	92%	126%	156%	130%	107%	31%	7%	14%	16%
PatchTST	Optimal	0.581	0.649	0.773	0.336	0.334	0.335	0.325	0.336	0.335	0.295	0.316	0.342	0.287	0.274	0.277
	Practice	0.611	0.718	0.864	0.407	0.474	0.509	0.369	0.482	0.592	0.294	0.315	0.341	0.276	0.267	0.278
	Δ_{MAE}	5%	11%	12%	21%	42%	52%	14%	43%	77%	0%	0%	0%	-4%	-2%	0%

Table 9: MSE when using Y_{t-H} and partial ground-truth.

Model	Method	Dataset H	ETTh2			ETTm1			Weather		
			24	48	96	24	48	96	24	48	96
PatchTST	GD		1.833	3.144	5.429	0.455	0.591	0.673	0.734	0.977	1.261
	PROCEED		1.703	3.070	5.299	0.420	0.570	0.650	0.719	0.967	1.256
iTransfo.	GD		2.463	3.759	6.152	0.459	0.602	0.692	0.851	1.131	1.406
	PROCEED		2.228	3.571	5.868	0.426	0.569	0.659	0.734	1.001	1.299

**Figure 7: Pipeline of PROCEED at each online time t . CDE: concept drift estimation; PMA: proactive model adaptation.**

is widely adopted by the state-of-the-art forecast models to reduce distribution shifts, lowering the forecasting loss in our experiments. As for the Optimal and Practical variants, we report the lowest errors when applying or not applying RevIN to FSNet and OneNet. Empirically, the Optimal variant without RevIN achieves better performance in some cases. For PROCEED, we search d_c in $\{50, 100, 150, 200\}$ and r in $\{24, 32, 48\}$. The MLP used in $\mathcal{E}$ consists of a linear layer with weights in $\mathbb{R}^{(L+H) \times d_c}$, a GeLU activation function, and a linear layer with weights in $\mathbb{R}^{d_c \times d_c}$, while the first linear layer of $\mathcal{E}'$ has weights in $\mathbb{R}^{L \times d_c}$.

C Pipeline of PROCEED

Fig. 7 depicts the pipeline of PROCEED during the online phase, which is a bit different from the steps introduced in Sec. 4.1. Given model parameters θ_{t-H-1} that have been updated with previous training data $\mathcal{D}_{(t-1)-}$, our model adapter additionally estimates the concept drift between $\mathcal{D}_{(t-1)-}$ and $\mathcal{D}_{t-}$ and adapts the model to $\mathcal{D}_{t-}$, yielding forecast feedback for updates. The reason behind it is that we jointly train the forecast model and the model adapter (see Sec. 4.4), while our training algorithm does not enforce the forecast model to perform well alone. Consequently, the forecast model without $\Delta\theta$ could forget some predictive skills, while the

parameter adjustments $\Delta\theta$ from the model adapter may incorporate the skills forgotten by the model. Thus it is necessary for the inference process to keep consistent with the training phase, *i.e.*, we always make predictions by integrating the forecast model and our model adapter as a whole. Nevertheless, in the feedback-based adaptation step, we only finetune the forecast model but keep the model adapter's parameters frozen. Otherwise, the model adapter may overfit the patterns of the drift between $\mathcal{D}_{(t-1)-}$ and $\mathcal{D}_{t-}$.

D Further Experimental Results

D.1 Performance Gap in terms of MAE

Table 8 shows the MAE errors of two variants of online learning introduced in Sec. 3. Likewise, we define Δ_{MAE} as the performance gap w.r.t. MAE. We can observe a general trend similar to Table 1, where a longer horizon can enlarge the performance gap between the Optimal and the Practical variants. The ECL and Traffic datasets do not have significant concept drift between each test sample and its preceding training sample. By contrast, Δ_{MSE} is more significant than Δ_{MAE} . We conjecture that online learning mainly benefits the forecast models in some rare or extreme cases of time series evolution where the frozen model's predictions deviate far from the ground truth and result in large MSE.

D.2 Performance with Partial Ground Truth

In this section, we introduce variants that update the model with feedback on Y_{t-H} and partial ground truth $\{\tilde{Y}_{t-i}\}_{i=1}^{H-1}$, where $\tilde{Y}_{t-i} = [\mathbf{v}_{t-i}, \dots, \mathbf{v}_{t-1}]$. Since each update involves H training samples, the GPU memory overhead becomes about H times than one involving Y_{t-H} only. We leave out results on ECL and Traffic datasets, where the GPU memory overhead exceeds the limit of our Nvidia 4090 GPUs (24GB). As shown in Table 9, PROCEED consistently outperforms the online gradient descent method under this setting. Compared with Table 3, using both Y_{t-H} and partial ground truth for feedback-based adaptation can improve the performance, while the improvement is insignificant. This indicates that samples with partial ground truth cannot reveal the concept of the test sample, necessitating proactive model adaptation. Though the partial ground truth can be beneficial to our method, the finetuning cost scales up with H (e.g., up to either $96\times$ latency or $96\times$ GPU memory). Thus, it would be much more efficient if using Y_{t-H} only. Notably, our proposed method using Y_{t-H} only can still outperform GD using H training samples in most cases.